

1. What is a Runnable in LangChain?

- A Runnable in LangChain is a standard interface that represents a component or unit of work that can process an input and produce an output.
- It allows components such as prompts, language models, output parsers, and custom functions to be executed individually or connected together to create workflows.

2. Why Do We Use Runnables in LangChain?

- We use Runnables in LangChain to execute individual components, connect multiple components, and build reusable workflows that process inputs and generate outputs efficiently.

Main purposes of Runnables

1. **Connect Components:** Connect prompts, language models, output parsers, and custom functions into a single workflow.
2. **Simplify Code:** Execute multiple steps using a simple and consistent interface.
3. **Create Sequential Workflows:** Execute tasks one after another, where the output of one step becomes the input of the next.
4. **Execute Parallel Tasks:** Perform multiple independent tasks using the same input.
5. **Reuse Components:** Use the same Runnable in different workflows.
6. **Process Multiple Inputs:** Use methods such as `.batch()` to process multiple inputs.
7. **Stream Outputs:** Use `.stream()` to receive output in chunks as it becomes available.
8. **Support Asynchronous Execution:** Use methods such as `.ainvoke()` for asynchronous processing.

3. Types of Runnables in LangChain

- 1) Task-Specific Runnables
- 2) Runnable Primitives

1) Task-Specific Runnables

- Prompt Runnable (ChatPromptTemplate) – Creates formatted prompts for the LLM.
- LLM Runnable (ChatGoogleGenerativeAI) – Generates responses using an AI model.
- Output Parser (StrOutputParser) – Converts model responses into plain text.
- Retriever Runnable – Retrieves relevant documents from a data source.

2) Runnable Primitives

- RunnableSequence – Executes steps one after another.
- RunnableParallel – Executes multiple independent tasks in parallel.
- RunnablePassthrough – Passes input without changing it.
- RunnableBranch – Choose a path based on a condition.

1] RunnableSequence

- RunnableSequence is a type of Runnable in LangChain that executes multiple components sequentially, where the output of one component becomes the input of the next component.
- Example :

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableSequence
```

```

model = ChatGoogleGenerativeAI(
    google_api_key="AQ.Ab8RN6KDoxeVy8BrTzxq3bot-Rai38jDLwzGtHqf2arrQWclwQ",
    model="gemini-3.8-flash"
)

detailed_prompt = ChatPromptTemplate.from_template(
    "write a proper professional one line title for blog where topic is {topic}"
)

parser = StrOutputParser()

chain = RunnableSequence(detailed_prompt , model , parser)

result = chain.invoke({
    "topic": "pm of india"
})

print(result)

```

2] RunnableParallel

- RunnableParallel is a type of Runnable in LangChain that executes multiple independent Runnables at the same time using the same input and returns all the results together.

```

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableSequence , RunnableParallel

model = ChatGoogleGenerativeAI(
    google_api_key="AQ.Ab8RN6KDoxeVy8BrTzxq3bot-Rai38jDLwzGtHqf2arrQWclwQ",
    model="gemini-3.8-flash"
)

prompt1 = ChatPromptTemplate.from_template(
    "write a one line title for blog where topic is {topic}"
)

```

```

)

prompt2 = ChatPromptTemplate.from_template(
    "write 5 seo friendly hashtags for blog where topic is {topic}"
)

parser = StrOutputParser()

parallel_chain = RunnableParallel({
    "title":RunnableSequence(prompt1 , model , parser),
    "tags":RunnableSequence(prompt2 , model , parser)
})

result = parallel_chain.invoke({
    "topic":"pm of india"
})

print(result)

```

3] RunnablePassthrough

- RunnablePassthrough is a Runnable in LangChain that passes the input directly to the next step without changing or processing it.

In simple words

- Input → Same Input → Next Step
- It is useful when we want to keep or pass the original input while also doing some other processing.

```

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough

model = ChatGoogleGenerativeAI(

```

```

    google_api_key="AQ.Ab8RN6KDoxeVy8BrTzxq3bot-Rai38jDLwzGtHqf2arrQWclwQ",
    model="gemini-3.8-flash"
)

passthrough = RunnablePassthrough()

result = passthrough.invoke("demo text")

print(result)

```

- If it gives the same output, why do we need it?
-
- RunnablePassthrough is not useful because it changes the input. It is useful because it lets us keep the original input while another Runnable processes it.
- Simple Scenario

We want to do two things:

- 1) Keep the original topic.
- 2) Generate a description of that topic using the LLM.

4] RunnableBranch

- RunnableBranch is a Runnable in LangChain that selects and executes one workflow from multiple possible workflows based on a given condition.
- In simple words
Think of it like an if-else statement.



YES	NO
↓	↓
Chain 1	Chain 2

Example

```
from langchain_google_genai import ChatGoogleGenerativeAI

from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

from langchain_core.runnables import RunnableSequence, RunnablePassthrough,
RunnableBranch

model = ChatGoogleGenerativeAI(
    google_api_key="AQ.Ab8RN6KDoxeVy8BrTzxq3bot-Rai38jDLwzGtHqf2arrQWclwQ",
    model="gemini-3.8-flash"
)

prompt1 = ChatPromptTemplate.from_template(
    "Write a proper, simple and short blog on {topic}"
)

prompt2 = ChatPromptTemplate.from_template(
    "Write a short summary of this blog:\n{content}"
)

parser = StrOutputParser()

# Blog generation
chain = RunnableSequence( prompt1, model, parser )

# Summary chain
summary_chain = RunnableSequence( prompt2, model, parser)
```

```
# Condition
def check_blog(blog):

    if len(blog.split()) > 500:
        return True

    else:
        return False

# Branch
branch_chain = RunnableBranch(
    (
        check_blog,
        summary_chain
    ),
    RunnablePassthrough()
)

# Final chain
final_chain = RunnableSequence(chain , branch_chain)

res = final_chain.invoke({
    "topic": "PM of India"
})

print(res)
```